

Deepening OOP: Transitioning to Data Structures

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Recap: The Blueprint vs. The Instance

Last class, we established that a **Class** is a blueprint, and an **Object** is a specific instance residing in memory.

```
class ConcordiaStudent:
    def __init__(self, name, student_id):
        # State (Attributes) belonging to the specific object
        self.name = name
        self.student_id = student_id
        self._gpa = 0.0 # Encapsulated private data

    def get_name(self):
        return self.name

# Creating INSTANCES (Concrete Objects)
alice = ConcordiaStudent("Alice", 101)
bob = ConcordiaStudent("Bob", 102)
charlie = ConcordiaStudent("Charlie", 103)
```

Now we have three distinct objects in memory. But how do we organize them?

The Problem with Raw Lists

Imagine the midterm exams have just finished. Alice, Bob, and Charlie hand their exams to the professor. We could put them in a standard Python list.

```
grading_pile = [alice, bob, charlie]

# A raw list allows ANY operation:
grading_pile.insert(0, charlie) # Shoving an exam into the middle
grading_pile.pop(1)             # Pulling an exam from the middle
grading_pile.reverse()          # Flipping the whole pile over
```

The Danger: Raw lists have no rules! Anyone can manipulate the data from any angle. In software engineering, when we want strict order and predictability, we must **encapsulate** the list inside a new Object.

Introducing the Stack (LIFO)

To bring order to our data, we introduce our first formal **Data Structure**: The Stack.

The LIFO Principle (Last-In, First-Out):

- Think of a physical pile of exams on a professor's desk.
- When Alice hands in her exam, it goes on the **top**.
- When Bob hands his in, it goes on **top** of Alice's.
- When the professor starts grading, they take the exam from the **top** of the pile (Bob's).
- You are *not allowed* to pull an exam from the bottom or the middle.

A Stack is Just Another Class!

We use our OOP knowledge to build the Stack. It is an Object whose entire purpose is to wrap a protective shield around a raw list.

```
class ExamStack:
    def __init__(self):
        # We use a raw list internally, but the underscore tells
        # other developers: "PRIVATE! Do not touch this directly!"
        self._pile = []

    # BEHAVIOR 1: Add to the top
    def push(self, student_exam):
        # .append() always adds to the very end (the "top")
        self._pile.append(student_exam)
        print(f"Exam for {student_exam.get_name()} added to the top.")
```

Notice how the *ExamStack* object is designed to hold *ConcordiaStudent* objects? This is called **Composition** (objects holding other objects)!

Enforcing the Rules (Encapsulation)

Now we add the methods to remove data. By only providing a `pop()` method, we mathematically guarantee that users cannot pull items from the middle.

```
class ExamStack:
    # ... (previous __init__ and push methods) ...

    # BEHAVIOR 2: Remove from the top
    def pop(self):
        if len(self._pile) == 0:
            return "The grading pile is empty!"

        # .pop() without arguments removes the LAST item in the list
        graded_exam = self._pile.pop()
        return graded_exam

    # BEHAVIOR 3: Just look at the top without removing it
    def peek(self):
        if len(self._pile) > 0:
            return self._pile[-1] # Index -1 is always the last item
        return None
```

Putting it all Together: Objects Interacting

Let's watch our ConcordiaStudent objects interact with our ExamStack object.

```
# 1. Create the instances
alice = ConcordiaStudent("Alice", 101)
bob = ConcordiaStudent("Bob", 102)

# 2. Create the Stack instance
prof_desk = ExamStack()

# 3. Students hand in exams (Pushing)
prof_desk.push(alice) # Alice is on the bottom
prof_desk.push(bob)   # Bob is on the top (Last-In)

# 4. Professor grades an exam (Popping)
current_exam = prof_desk.pop() # First-Out

print(f"The professor is grading: {current_exam.get_name()}")
# Output: The professor is grading: Bob
```

Why did we use OOP for this?

The Power of Encapsulation:

- If we gave the professor the raw list (`_pile`), they could accidentally run `_pile.clear()` and delete all the exams!
- By burying the list inside the `ExamStack` class, we **abstract** away the list mechanics.
- The user is forced to use `.push()` and `.pop()`, ensuring the LIFO (Last-In, First-Out) rule is never broken.

Looking Ahead to Lab 9: In today's lab, you won't be pushing `Student` objects. You will be pushing opening parentheses (`{` `[` onto a stack to ensure mathematical equations are properly closed in reverse order!

Questions?

Get ready for Lab 9: Validating Parentheses using a Stack!